Date : 19 March 2026

Antoine Zoghbi
(68 Rue de Mingan, Gatineau, Québec, Canada, J8R2C9)

Algorithm Z1
Accelerated and parallel Generation of Integer Partitions in Standard form

Improving the Fastest Existing Algorithm ZS1

Version 2.1

Abstract

Algorithm ZS1, for generating integer partitions in standard anti-lexicographic order, was originally presented in Mr. Zoghbi's Master thesis "Algorithms for Generating Integer Partitions" (University of Ottawa, 1993) and later published in "Fast Algorithms for Generating Integer Partitions" (Zoghbi & Stojmenović, International Journal of Computing Mathematics, 1998, Vol. 70, pp. 319–332). It is widely regarded by specialists as representing the state-of-the-art in this area. Algorithm Z1 is an optimized version of ZS1. Performance evaluations demonstrate that Z1 achieves an execution-time reduction of approximately 36% relative to ZS1. Multi-threaded execution of ZS1 and Z1 achieved up to an 89% reduction in runtime compared to sequential execution, highlighting the strong dependence of performance gains on compiler behavior and system architecture.

Table of Contents

Contents

1. Introduction 3

2. Partitions with parts 2 and 3 4

3. Algorithms ZS1 / Z1 4

3.1 Algorithm ZS1 4

3.2 Algorithm Z1 5

4. Generating Balanced Distribution Partitions 8

4.1 Algorithm Z3, 8 Threads, *n*=150 8

4.2 Hardware Capacity 9

5. Multi-Threaded Parallel Execution on the Same Hardware System 10

5.1 Algorithm ZS1, 16 Threads, *n*=170 10

5.2 Algorithm Z1, 16 Threads, *n*=170 11

6. Summary 12

7. Conclusions 13

8. Conclusions: Algorithm Z1 vs Existing Parallel Algorithms 14

9. Open Discussion and challenges 14

10. Standard Form Partitions 15

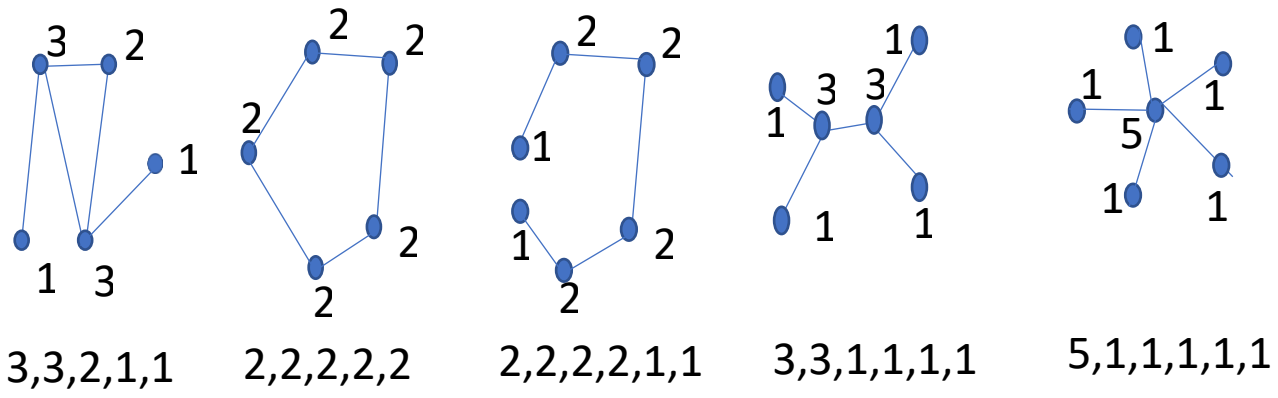
11. Algorithms : ZS1 / Z1 / Z3 – C codes 16

}**12. References:** 20

1. Introduction

Mr. Zoghbi's Master's thesis, *Algorithms for Generating Integer Partitions* (University of Ottawa, 1993), and the subsequent paper *Fast Algorithms for Generating Integer Partitions* (Zoghbi & Stojmenović, *International Journal of Computer Mathematics*, 1998), introduced the ZS1 and ZS2 algorithms. These methods quickly became recognized for their simplicity, efficiency, and strong performance. Algorithm ZS1, in particular, has been adopted across a wide range of fields, including networking, agriculture, music, healthcare (such as DNA analysis), civil engineering, and aerospace, with additional references listed in Section 12.

The network graphs below illustrate representative partition structures of the integer 10, where nodes denote parts and edges reflect their values. Only a subset of the full partition set is shown for clarity.



The recently published work *Algorithm Z1: Accelerated and Parallel Generation of Integer Partitions in Standard Form* introduced Algorithm Z1 (formerly AZ1) in anti-lexicographic order and demonstrated performance improvements of up to 85% compared with ZS1.

Version 2.0 further enhances the algorithm by incorporating:

- A direct-concatenation method for cases where the rightmost part equals 3, removing unnecessary intermediate steps and reducing computation.
- Algorithm Z3, which generates fairly distributed and balanced partition subsets.
- A batch-processing mechanism designed to automatically initiate and manage parallel threads.

Algorithms ZS1 and Z1 were evaluated under a single-system, multi-threaded execution model in which the partition space was divided into disjoint subsets according to ranges of the largest part.

Parallel execution of these subsets achieved runtime reductions of up to 89% compared to sequential execution.

Algorithm Z2 (Formerly AZ2) was introduced in the earlier version, and it is not discussed in this work,

All reported performance measurements exclude output operations.

2. Partitions with parts 2 and 3

The number of integer partitions $P(n)$ increases exponentially with n. As shown in the accompanying table, the proportion of partitions whose leftmost part is 2 or 3 followed, when applicable, by a sequence of 1s also grows steadily as n increases.

Integer	# of partitions P(n)	Count_2	Count_2&3	% Count_2	% Count_2&3
10	42	22	28	52.38%	66.67%
50	204,226	147,273	164,215	72.11%	80.41%
100	190,569,292	150,198,136	160,879,829	78.82%	84.42%
120	1,844,349,560	1,482,074,143	1,574,166,343	80.36%	85.35%
150	40,853,235,313	33,549,419,497	35,308,939,917	82.12%	86.43%
160	107,438,159,466	88,751,778,802	93,179,849,732	82.61%	86.73%
170	274,768,617,130	228,204,732,751	239,065,425,095	83.05%	87.01%
190	1,667,727,404,093	1,398,341,745,571	1,459,318,607,756	83.85%	87.50%
200	3,972,999,029,388	3,345,365,983,698	3,485,429,644,438	84.20%	87.73%

Table 1

3. Algorithms ZS1 / Z1

3.1 Algorithm ZS1

```
Input integer -> n;
Input lowest value of leftmost part -> x1;
Input highest value of leftmost part -> x2;
If (x1 = 1) then {x1 ← 2};
For I ←0 to n do a[i] ←1;
A[1] ←x2+1; h←1; m←n – x2; x←0;
A[1] ←n; h←1; m←1; x←0;
While(a[1]>=x1)  do {
                    If (a[h] = 2) then {m←m+1, a[h] ←1; h←h-1;}
                   else {
                        t ← m – h + 1; a[h] ← a[h] – 1; r ←a[h];
                        while (t >= r) do {h ← h+1; a[h] ← r; t ← t – r;}
                        if  (t <2) then {m ← h + t;}
                        else { m ← h + 1; h ← h + 1; a[h] ← t;}
                        }
                       if (a[1] >= x1) then {for  i ← 1 to m do output a[i];}
                   }
if (a[1] == 1) then {for  i ← 1 to n do output a[i];}
```

Algorithm ZS1 was extended to allow users to specify explicit lower and upper bounds on the leftmost part of the partition, thereby restricting the generation process to a designated subset of the integer partitions of n. The algorithm processes partition whose rightmost non-unit part equals 2 with particular efficiency: in this case, the successor partition is produced using only three elementary operations. This behavior confirms that ZS1 satisfies a **constant-average-delay** property, as established in "*Fast Algorithms for Generating Integer Partitions*".

Consequently, ZS1 remains the fastest known algorithm within its class and is widely used. Moreover, as n increases, the proportion of partitions that contain at least one part equal to 2 typically followed by a block of ones grows significantly, as shown in Table 1.

- For $n = 50$, 72.11% of the partitions end with a 2 followed by ones (if any).
- For $n = 170$, this proportion increases to 83.05%.
- For $n = 190$, this proportion increases to 83.85%.

Accordingly, as n increases, a progressively larger proportion of partitions falls into this class, accounting for the strong empirical performance and scalability of algorithm ZS1.

3.2 Algorithm Z1

Algorithm Z1 is an optimized variant of ZS1 designed to accelerate the generation of integer partitions in anti-lexicographic order. It supports explicit bounds on the leftmost part, enabling restricted-range generation without modifying the core algorithm. Its primary improvement is a specialized fast-path for partitions whose rightmost part is 3 (optionally followed by ones), eliminating the iterative redistribution loop used in ZS1.

Z1 processes such partitions by directly redistributing the rightmost 3 and propagating the transformation until all 3s and any intermediate 2s are reduced to 1s. For example, for $n = 10$, the partition $(4, 3, 3)$ triggers case $x = 0$, producing the sequence:

$$(4,3,2,2),\ (4,3,2,1,1),\ (4,3,1,1,1).$$

In the final partition, the trailing 3 reappears immediately before the block of 1s, invoking case $x = 3$. Z1 then directly emits:

$$(4,2,2,2),\ (4,2,2,1,1),\ (4,2,1,1,1,1),\ (4,1,1,1,1,1,1).$$

Algorithm Z1

```
Input integer -> n;
Input lowest value of leftmost part -> x1;
Input highest value of leftmost part -> x2;
If (x1 = 1) then {x1 ← 2;};
For I ←0 to n do a[i] ←1;
A[1] ←x2+1; h←1; m←n – x2; x←0;
While(a[1]>=x1)  do {
                    While(a[h]=2) do {
                                m←m+1, a[h] ←1; h←h-1;
                                for  i ← 1 to m do output a[i];
                                }
                    if (a[h] = 3) then {
                                x ← m – h; a[h] ←2;
                                if (x > 3) then {
                                            while (x > 0) do {h←h+1; a[h] ← 2; x ← x – 2;}
                                            if (x=0) then {m ← h + 1;}
                                        else {m ← h;}
                                            }
                        else if ( x = 3) then {
                                            for  i ← 1 to h do output a[i]; output " 2 2";
                                            for  i ← 1 to h do output a[i]; output " 2 1 1";
                                            for  i ← 1 to h do output a[i]; output " 1 1 1 1";
                                            a[h] ← 1; h ← h – 1; m ← m + 2;}
                        else if ( x = 1) then {
                                            for  i ← 1 to h do output a[i]; output " 2";
                                            for  i ← 1 to h do output a[i]; output " 1 1";
                                            a[h] ← 1; h ← h – 1; m ← m + 2;}
                        else if (x =2)   then {
                                            for  i ← 1 to h do output a[i]; output " 2 1";
                                            for  i ← 1 to h do output a[i]; output " 1 1 1";
                                            a[h] ← 1; h ← h – 1; m ← m + 2;}
                                    else  { /* x = 0 and m=h */
                                            m ← m + 1;
                                            for  i ← 1 to m do output a[i];
                                            a[h] ← 1; h ← h – 1; m ← m + 1;
                                            }
                                // for all cases x=0, x=1, x=2, x=3
                                for  i ← 1 to m do output a[i];
                                } /* a[h] =3 */
                else if (a[h] > 3) then {
                                t ← m – h + 1; a[h] ← a[h] – 1; r ←a[h]
                                while (t >= r) do {h ← h+1; a[h] ← r; t ← t – r}
                                if  (t <2) then {m ← h + t;}
                                else { m ← h + 1; h ← h + 1; a[h] ← t;}
                                for  i ← 1 to m do output a[i];
                                } /* a[h] > 3 */
            } /* while a[1] >= x1 */
```

Each output partition is constructed by concatenating a fixed prefix with suffixes obtained from redistributing the rightmost 3. For instance, from $(4, 3, 2, 2)$, the prefix $(4, 3)$ is combined with all partitions of 4 derived from $(2, 2)$, such as $(2, 1, 1)$ and $(1, 1, 1, 1)$, yielding:

$$(4,3,2,1,1),\ (4,3,1,1,1,1).$$

This direct-concatenation mechanism avoids repeated state reconstruction and eliminates several inner-loop transitions, providing an additional ~5% efficiency gain and raising overall improvement to approximately 89%.

By explicitly handling rightmost 3 structures, Z1 removes the most expensive iterative steps in ZS1, thereby achieving substantially lower average delay and improved empirical performance

Algorithm Z1 accelerates generation by optimizing the handling of partitions whose rightmost structure consists of a 2 or 3 followed by any number of 1s.

For ***n* = 170**, Algorithm Z1 produces:

- **207,202,977,471** partitions where the rightmost part is **2** before the block of 1s.
- **31,862,447,624** partitions where the rightmost part is **3** before the block of 1s.

In comparison, Algorithm ZS1 generates **228,204,732,751** such partitions ending in 2. The difference **228,204,732,751 − 207,202,977,471 = 21,001,755,280** represents partitions where a 2 is produced solely from a rightmost 3. Algorithm Z1 processes these cases in batches whenever a rightmost 3 is encountered, improving efficiency.

For ***n* = 150**, Z1 increases the proportion of special-case partitions from **82.12%** (ZS1) to **86.43%** (Z1).
For ***n* = 170**, this proportion rises from **83.05%** to **87.01%**.
These results show that as *n* grows, partitions with rightmost parts equal to 2 or 3 constitute an increasingly large share of all partitions.

Performance tests were conducted using the MSYS MinGW64 compiler on an AMD Ryzen™ 7 5800HS system (8 cores, 16 threads, 16 GB RAM). For *n* = 170, algorithm ZS1 required 1099.354004 seconds in a sequential run, whereas algorithm Z1 completed the same task in 696.825012 seconds a 36.62% reduction in execution time.

AMD Ryzen 7 5800HS,8 cores, 16 processors, 16 RAM				
Integer	ZS1 Time Sec	Z1 Time Sec	Z1 Time Reduction (Sec)	Z1 Time Saving Sec %
10	0.000000	0.000000	0.00%	0.00%
50	0.000000	0.000000	0.00%	0.00%
100	0.734000	0.510000	69.48%	30.52%
150	151.744003	105.403000	69.46%	30.54%
170	1099.354004	696.825012	63.38%	36.62%

When evaluated using the MSYS MinGW64 compiler on an Intel™ Core i7-12700H (2.3 GHz), performance tests for $n = 150$ algorithm ZS1 required 522.903992 seconds in a sequential run, whereas, algorithm Z1 completed the same task in 297.903992 seconds.

These findings indicate that Z1's performance advantages are both hardware and compiler-dependent.

4. Generating Balanced Distribution Partitions

4.1 Algorithm Z3, 8 Threads, $n = 150$

Algorithm Z3, a refinement of Z1, produces a balanced distribution of partition subsets by:

- Dividing the partitions into disjoint subsets.
- Ensuring all partitions sharing the same leftmost part are assigned to a single subset.
- Structuring subsets to support efficient parallel or distributed execution.

Example of Z3 input/output:

```
C:\msys64\mingw64\code\Z3

Algorithm Z3 - Multi-Thread setting

Please enter the value of integer N : 150

Please enter the lowest  value of the leftmost part: 1

Please enter the highest value of the leftmost part: 150

Please enter the Multi-Threads: 8

Please enter the Partitions factor: 1.3
----------------- Finding Threads with Partitions Borders  -----------------

PN: 40853235313        cpu_time: 0.000000 s  (CLOCKS_PER_SEC:1000)

Maximum Partitions per one Thread 6638650368

Thread - 1  -  Lower  39  -  Higer  150  -  Partitions: 6638650369    cpu_time : 20.044001 s

Thread - 2  -  Lower  33  -  Higer  38  -  Partitions: 6638650370    cpu_time : 39.639999 s

Thread - 3  -  Lower  29  -  Higer  32  -  Partitions: 6638650387    cpu_time : 59.287998 s

Thread - 4  -  Lower  26  -  Higer  28  -  Partitions: 6638650387    cpu_time : 78.598000 s

Thread - 5  -  Lower  23  -  Higer  25  -  Partitions: 6638650372    cpu_time : 97.750000 s

Thread - 6  -  Lower  20  -  Higer  22  -  Partitions: 6638650376    cpu_time : 117.061996 s

Thread - 7  -  Lower  14  -  Higer  19  -  Partitions: 6638650370    cpu_time : 135.772003 s

Thread - 8  -  Lower  1  -  Higer  13  -  Partitions: 927015437     cpu_time : 138.199997 s

Start  : Fri Feb 06 20:52:26 2026

Finish : Fri Feb 06 20:54:45 2026
```

The *Partitions Factor* parameter improves load balancing by ensuring a more even distribution of partitions across subsets. Each subset is assigned based on the total number of partitions and the number of available threads. Since all partitions with the same leftmost part must remain together, any subset boundary that would split such a block instead assigns the entire block to the next subset.

To support efficient parallel execution, Algorithms Z1 and ZS1 were adapted to accept input parameters directly from a batch file.

```
int main(int arg1, char *argv[]) {
   n = atoi (argv[3]); x1 = atoi (argv[4]); x2 = atoi (argv[5]);
```

as an example in the batch file Z1a190.16.bat below: n=190, x1=28, x2=29 etc….

Example of batch file:

Batch file name: Z1a190.16.bat

```
echo Starting launching All scripts in parallel
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 28 29
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 26 27
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 31 32
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 23 24
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 52 190
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 33 34
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 39 41
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 42 45
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 46 51
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 18 20
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 21 22
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 35 36
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 37 38
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 25 25
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 30 30
start C:/msys64/mingw64/code/Z1a 3 Z1a 190 1  17
echo Finish launching All scripts in parallel
```

Tasks are prioritized by partition count and estimated execution time, with larger workloads scheduled first to minimize Startup imbalance and ensure a more even distribution of computational load across threads

4.2 Hardware Capacity

Algorithm Z3 computes *P*(n) using a single array. For example, for *n* = 200, it yields ***P*(200) = 3,972,999,029,388**. However, due to hardware and compiler limitations, generating all partitions for this value in a single sequential run is impractical. Empirical testing on an **AMD Ryzen™ 7 5800HS** system (8 physical cores, 16 logical processors, 16 GB RAM) indicates that the largest value that can be reliably managed on a single system is approximately *n* = 170. This limit, however, depends on the available hardware.

For ***n* = 190**, using **Algorithm ZS1**, a full single-run generation ultimately caused system instability. Sequential execution of individual subsets was successful (total time: 6139 seconds), but parallel execution of all subsets was not feasible.

For **Algorithm Z1** at ***n* = 190**, a complete single-run execution also proved unstable. Nonetheless, sequential subset execution completed successfully in 4174 seconds, and parallel execution of all subsets was achievable representing a 32% reduction in total time compared to ZS1 (4174 vs. 6139 seconds).

For ***n* = 200**, using **Algorithm ZS1**, generating all partitions in a single run was not possible. While sequential execution of individual subsets was feasible, full parallel execution again resulted in instability.

For **Algorithm Z1** at ***n* = 200**, single-run generation was not achievable, but sequential execution of subsets remained stable and delivered roughly a 34% performance improvement over ZS1. For example, subset 60–200 required 596 seconds under ZS1 and 393 seconds under Z1. Full parallel execution of all subsets in one batch was not stable; therefore, parallel processing was conducted across three batches, with occasional instability still observed.

5. Multi-Threaded Parallel Execution on the Same Hardware System

Performance for algorithms **ZS1** and **Z1** was evaluated under multi-threaded execution, where multiple instances of each program were run concurrently, with each instance generating a distinct subset of the integer-partition space. This approach yielded execution-time reductions of up to **89%**.

All experiments were performed using the **MSYS MinGW64** compiler on an **AMD Ryzen™ 7 5800HS** system (8 physical cores, 16 logical processors, 16 GB RAM). Partition subsets were distributed across threads through iterative tuning to achieve near-optimal workload balance.

5.1 Algorithm ZS1, 16 Threads, *n*=170

For *n = 170*, sixteen-thread execution reduced runtime from **1099.354 s** (sequential) to **157.539 s**, representing **14.33%** of the original runtime—a reduction of **85.67%**. Average CPU utilization during parallel execution was **97%**.

AMD Ryzen 7 5800HS system CPU usage 97%							
					Multi-Threaded Parallel execution (single hardware)		
n=170	Partitions from	Partitions To	Number of Partitions P(n)	Time Sec	Time Sec	Time Start	Time End
ZS1	1	170	274,768,617,130	1099.354004			
ZS1	49	170	19,193,773,961	74.348999	140.298004	8:27:10	8:29:31
ZS1	43	48	17,659,663,004	67.250000	127.806999	8:27:10	8:29:18
ZS1	39	42	19,035,618,336	71.633003	138.615000	8:27:10	8:29:29
ZS1	36	38	19,456,938,801	72.300003	140.289993	8:27:10	8:29:30
ZS1	34	35	15,826,760,953	58.849998	116.635002	8:27:15	8:29:11
ZS1	32	33	18,218,831,388	67.400002	133.645004	8:27:10	8:29:24
ZS1	30	31	20,511,517,482	75.767998	147.029007	8:27:10	8:29:36
ZS1	28	29	22,438,541,109	82.203003	**157.539993**	**8:27:09**	**8:29:47**
ZS1	27	27	11,732,853,219	42.695999	88.816000	8:27:15	8:28:43
ZS1	26	26	11,913,686,128	43.589001	90.138000	8:27:15	8:28:45
ZS1	25	25	11,941,084,517	43.752998	90.412003	8:27:15	9:28:45
ZS1	24	24	11,791,495,467	42.509998	89.194000	8:27:15	8:28:44
ZS1	22	23	22,340,653,336	81.047997	156.395996	8:27:10	8:29:46
ZS1	20	21	19,325,817,959	69.013000	138.604004	8:27:10	8:29:28
ZS1	17	19	20,544,577,090	71.592003	144.625000	8:27:10	8:29:34
ZS1	1	16	12,836,804,380	43.292999	92.642998	8:27:15	8:28:48
TOTAL			274,768,617,130	1007.247001	**157.539993**	**8:27:09**	**8:29:47**

5.2 Algorithm Z1, 16 Threads, *n*=170

Under identical experimental conditions, Algorithm Z1 was executed using sixteen parallel threads on the same hardware platform, with each thread responsible for generating a unique subset of the integer-partition space. The parallel run completed in 124.427 s, compared with 696.825 s for the sequential execution. This represents 17.86% of the sequential runtime, corresponding to a 82.14% reduction in total execution time. During multi-threaded execution, the system sustained an average CPU utilization of 97%.

AMD Ryzen 7 5800HS system CPU usage 97%							
					Multi-Threaded Parallel execution (single hardware)		
n=170	Partitions from	Partitions To	Number of Partitions P(n)	Time Sec	Time Sec	Time Start	Time End
Z1	1	170	274,768,617,130	696.825012			
Z1	49	170	19,193,773,961	51.344002	113.330002	8:15:05	8:16:58
Z1	43	48	17,659,663,004	46.152000	101.664001	8:15:05	8:16:49
Z1	39	42	19,035,618,336	49.728001	111.119003	8:15:10	8:17:01
Z1	36	38	19,456,938,801	50.187000	113.459999	8:15:05	8:16:58
Z1	34	35	15,826,760,953	40.425999	94.254997	8:15:05	8:16:39
Z1	32	33	18,218,831,388	46.551998	105.808998	8:15:05	8:16:51
Z1	30	31	20,511,517,482	53.355000	119.105003	8:15:05	8:17:04
Z1	28	29	22,438,541,109	56.541000	**124.427002**	**8:15:05**	**8:17:09**
Z1	27	27	11,732,853,219	29.278999	71.709000	8:15:05	8:16:17
Z1	26	26	11,913,686,128	29.958000	72.875000	8:15:06	8:16:19
Z1	25	25	11,941,084,517	29.827999	72.261002	8:15:06	8:16:18
Z1	24	24	11,791,495,467	29.166000	69.338000	8:15:06	8:16:15
Z1	22	23	22,340,653,336	54.713001	119.942000	8:15:10	8:17:09
Z1	20	21	19,325,817,959	47.192000	112.999000	8:15:10	8:17:03
Z1	17	19	20,544,577,090	48.462000	115.941002	8:15:11	8:17:07
Z1	1	16	12,836,804,380	29.027000	76.518000	8:15:11	8:16:27
TOTAL			274,768,617,130	691.909999	**124.427002**	**8:15:05**	**8:17:09**

Under sixteen-thread parallel execution on the same hardware platform, Algorithm Z1 achieved 78.98% of the runtime of Algorithm ZS1 (124.427002 s vs. 157.539993 s), reflecting a 21.02% additional performance gain.

In total, the parallel execution reduced runtime to 11.32% of the original sequential ZS1 execution time (124.427 s vs. 1099.354 s), corresponding to an 88.68% overall reduction. Average CPU utilization during the parallel run was 97%.

6. Summary

The performance evaluation of the proposed algorithms is presented in the accompanying table, which reports sequential and multi-threaded execution times, percentage reductions in runtime, estimated time savings, and average CPU utilization across multiple values of *n* and thread counts. Across all experiments, Algorithm Z1 consistently outperforms ZS1 under both sequential and parallel execution conditions. For example, reducing the sequential execution time from 696.825 s to 124.427 s corresponds to an 82.14% decrease, while comparing 124.427002 s to the original ZS1 baseline of 1099.354 s yields an overall reduction of 88.68%. These results demonstrate the superior efficiency and scalability of Algorithm Z1 relative to ZS1.

AMD Ryzen 7 5800HS system							
				Sequential	Multi-Threaded Parallel execution (single hardware)		
Algorithm	n	Parallel Runs	Single Execution Time Sec/ Sec per Partition	Runs Time Sec / Sec per Partition	Time Sec / Sec per Partition	Time Saving	CPU usage
ZS1	150	4	154.584002	154.434006	62.945999	59.28%	28%
			3.783886E-09	3.780215E-09	1.540784E-09		
Z1	150	4	108.193001	106.355001	47.875999	55.75%	28%
			2.648334E-09	2.603343E-09	1.171902E-09	69.03%	
ZS1	150	17	151.744003	150.232000	23.315001	84.64%	97%
			3.714369E-09	3.677359E-09	5.707015E-10		
Z1	150	17	105.403000	103.927000	18.730000	82.23%	97%
			2.580040E-09	2.543911E-09	4.584704E-10	87.66%	
ZS1	170	16	1099.354004	1007.247001	157.539993	85.67%	97%
			4.001017E-09	3.665801E-09	5.733551E-10		
Z1	170	16	696.825012	691.909999	124.427002	82.14%	97%
			2.536043E-09	2.518155E-09	4.528428E-10	88.68%	
ZS1	190	16	0.000000	6250.526956	0.000000	0.00%	97%
			0.000000E+00	3.747931E-09	0.000000E+00		
Z1	190	16	4174.296875	4127.428999	763.338939	81.71%	97%
			2.502985E-09	2.474882E-09	4.577121E-10	87.79%	

7. Conclusions

Algorithm Z1 consistently outperforms ZS1 in both sequential and parallel execution settings. Although both algorithms maintain the constant average-delay property, their practical performance is influenced by system architecture and compiler behavior. The performance advantage of Algorithm Z1 becomes particularly pronounced for values of $n > 100$.

Empirical results demonstrate that parallel execution on a single hardware system provides substantial runtime reductions. Using four threads yields an approximate 69% reduction in execution time, while sixteen threads achieve reductions of nearly 89%. Notably, subdividing the generation process enhances performance even when parallelism is not employed.

Execution of a single subset exhibits better performance than concurrent execution of multiple subsets, as parallel launches introduce overhead that increases with the number of simultaneously executed subsets. A key advantage of ZS1 and Z1 is that they support selective generation of partitions through parameter adjustment alone, without requiring structural modifications to the algorithms. This characteristic differentiates them from alternative standard-form generation methods, particularly those outside the anti-lexicographic and lexicographic categories, which tend to rely on additional conditional checks that significantly increase computational overhead.

Another important observation is that generating the full partition set through multiple sequential subset runs is faster than performing a single monolithic run. For example, when $n = 170$, a complete execution of Algorithm Z1 required 696.825 seconds, whereas the sequential execution of sixteen subdivided runs required a total of 691.909 seconds. This pattern, observed consistently across experiments, suggests that subdivision reduces system load and improves performance, even in the absence of parallel execution.

The ability to execute subsets sequentially or in parallel and subsequently combine their outputs makes it feasible to generate partitions for significantly larger values of n than would be practical in a single run. For instance, this approach enables full computation of partitions for $n = 190$ using Algorithm ZS1.

For sufficiently large n, a trade-off emerges between executing subsets sequentially or in parallel on a single system and distributing them across multiple hardware systems, each responsible for a distinct subset. The case $n = 200$ illustrates these considerations and highlights the resource limitations inherent in large-scale partition generation.

For large *n*, the most efficient approach to generating all partitions is to distribute subsets across multiple machines, ensuring that the number of subsets does not exceed the threads available on each machine and that total CPU usage on each machine stays below 97%.

All performance measurements reported in this study were obtained with output operations disabled.

8. Conclusions: Algorithm Z1 vs Existing Parallel Algorithms

Theoretical analysis shows that Algorithm Z1 functions as a high-efficiency sequential baseline, characterized by strong per-core computational throughput and minimal control-flow overhead. These properties enable it to remain competitive with parallel partition-generation methods, particularly in regimes where synchronization, communication, and thread-management overhead dominate total execution cost.

8.1 Cross-Model Comparative Summary

Criterion	Algorithm Z1	Distributed Parallel	Mutli-Threaded Parallel
Per-Core Efficiency	Very High	Low-Moderate	Moderate
Absolute Throughput	Low-Moderate	Very High	High
Scalability	Low	Very High	Moderate
Execution Overhead	Minimal	High	Moderate
Implementation Complexity	Low	High	Moderate

8.2 Framework Conclusion

Algorithm Z1 provides a highly efficient sequential baseline, characterized by strong runtime performance and low execution overhead. Although certain parallel algorithms may achieve greater throughput or scalability, these gains often come at the cost of increased coordination, communication, and synchronization overhead. Consequently, the practical benefits of parallelism depend strongly on the structure and characteristics of the workload. Within this context, Algorithm Z1 remains a competitive option, particularly in environments where overhead-related constraints diminish the effectiveness of parallel approaches.

9. Open Discussion and challenges

Develop an algorithm for generating all integer partitions of large n on a single hardware system. For $n = 500$, $P(500) = 3{,}991{,}268{,}667{,}606{,}861{,}086{,}720$, necessitating an efficient enumeration strategy. The method employs a structured subdivision scheme, partitioning the solution space into subsets arranged vertically and horizontally following the Z1 approach. Each complete partition is constructed by concatenating component partitions from these subsets. The independence of subsets enables parallel execution and supports distribution across multiple hardware systems, thereby enhancing scalability and computational efficiency.

For $n = 170$, executing 16 subsets on a 16-thread processor reduced computation time by 89% with CPU utilization below 98%. Similar tests on 32- and 64-thread systems using 32 and 64 subsets, respectively, are expected to exceed 89% speedup while maintaining CPU utilization under 98%.

A comparative evaluation framework should also be developed to assess Algorithm Z1 against existing parallel algorithms. The evaluation should consider performance, scalability, runtime efficiency, and execution overhead under two deployment models:

- Distributed or multi-system parallelism
- Single-system, multi-threaded parallelism.

10. Standard Form Partitions

For n = 10, the Standard form of the Partitions are:

Antilexicografic order											Lexicographic order									
ZS1 / Z1											ZS2 / Z2									
10											1	1	1	1	1	1	1	1	1	1
9	1										2	1	1	1	1	1	1	1	1	
8	2										2	2	1	1	1	1	1	1		
8	1	1									2	2	2	1	1	1	1			
7	3										2	2	2	2	1	1				
7	2	1									2	2	2	2	2					
7	1	1	1								3	1	1	1	1	1	1	1		
6	4										3	2	1	1	1	1	1			
6	3	1									3	2	2	1	1	1				
6	2	2									3	2	2	2	2					
6	2	1	1								3	3	1	1	1	1				
6	1	1	1	1							3	3	2	1	1					
5	5										3	3	2	2						
5	4	1									3	3	3	1						
5	3	2									4	1	1	1	1	1	1			
5	3	1	1								4	2	1	1	1	1				
5	2	2	1								4	2	2	1	1					
5	2	1	1	1							4	2	2	2						
5	1	1	1	1	1						4	3	1	1	1					
4	4	2									4	3	2	1						
4	4	1	1								4	3	3							
4	3	3									4	4	1	1						
4	3	2	1								4	4	2							
4	3	1	1	1							5	1	1	1	1	1				
4	2	2	2								5	2	1	1	1					
4	2	2	1	1							5	2	2	1						
4	2	1	1	1	1						5	3	1	1						
4	1	1	1	1	1	1					5	3	2							
3	3	3	1								5	4	1							
3	3	2	2								5	5								
3	3	2	1	1							6	1	1	1	1					
3	3	1	1	1	1						6	2	1	1						
3	2	2	2	1							6	2	2							
3	2	2	1	1	1						6	3	1							
3	2	1	1	1	1	1					6	4								
3	1	1	1	1	1	1	1				7	1	1	1						
2	2	2	2	2							7	2	1							
2	2	2	2	1	1						7	3								
2	2	2	1	1	1	1					8	1	1							
2	2	1	1	1	1	1	1				8	2								
2	1	1	1	1	1	1	1	1			9	1								
1	1	1	1	1	1	1	1	1	1		10									

11. Algorithms : ZS1 / Z1 / Z3 – C codes

ZS1 - C code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int m,n,i,h,t,r,x1,x2,y1,y2,z1,z2,z3;
long long int v,w;
clock_t clock1, clock2;
time_t tstart, tfinish;
float cpu_time_used;

int main(void) {
   printf("\n Algorithm ZS1 - Restricted \n \n \n ");
   printf("\n please enter the value of integer N : ");
   scanf ("%d", &n);
   printf("\n please enter the lowest  value of the leftmost part: ");
   scanf ("%d", &x1);
   printf("\n please enter the highest value of the leftmost part: ");
   scanf ("%d", &x2);
   if (x1 == 1) {x1 = 2;}
   printf("\n \n \n Generating Standard Antilexicographic Partitions for integer %d \n \n", n);

   int a[n+1];
   for (i=0; i<=n; ++i) a[i]=1;
   a[1] = x2 + 1; h = 1; m = n - x2;

   time(&tstart);
   printf("\n Start  : %s", ctime(&tstart));
   clock1 = clock();

   while (a[1] >= x1){
          if     (a[h] == 2) { m += 1; a[h] = 1; --h; }
          else {
                 t = m-h+1; a[h] -=1; r = a[h];
                while (t >= r) { h +=1; a[h] = r; t -=r;}
                     if     (t < 2) {m = h + t;}
                else {m = h + 1; ++h; a[h] = t;}
                }
           if (a[1] >= x1) {for(i = 1; i <= m; ++i) (printf(" %d", a[i])); printf(" \n");}
           } // while (a[1] >= x1)

    if (a[1] == 1) {for(i = 1; i <= n; ++i) (printf(" %d", a[i])); printf(" \n");}
    clock2 = clock();
    cpu_time_used = ( (float) (clock2-clock1))/CLOCKS_PER_SEC;
    printf("\n cpu_time_used : %lf seconds \n", cpu_time_used);
    printf("  ( CLOCKS_PER_SEC:%d) \n",CLOCKS_PER_SEC);
    printf("\n Start  : %s", ctime(&tstart));
    time(&tfinish);
    printf("\n Finish : %s", ctime(&tfinish));
    scanf ("%d", &i);
    return 0;
}
```

Z1 – C code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

int m,n,i,h,t,r,x,x1,x2;
long long int v,v0,v1,v2,v3,v23,v4,w,y,t0,t1,t2;
clock_t clock1, clock2;
time_t tstart, tfinish;
float cpu_time_used;
float vp=0.00;
int main(void) {
   printf("\n Algorithm Z1 - Restricted \n \n \n ");
   printf("\n please enter the value of integer N : ");
   scanf ("%d", &n);
   printf("\n please enter leftmost part from: ");
   scanf ("%d", &x1);
   printf("\n please enter leftmost part to  : ");
   scanf ("%d", &x2);
   if (x1 == 1) {x1 = 2;}
   printf("\n \n \n Generating Standard Antilexicographic Partitions for integer %d \n \n", n);
   int a[n+1];
   for (i=0; i<=n; ++i) a[i]=1;
   a[1] = x2 + 1; h = 1; m = n - x2; w=0;
   time(&tstart);
   printf("\n Start  : %s", ctime(&tstart));
   clock1 = clock();

   while (a[1] >= x1){
            while (a[h] == 2){
                    ++m; a[h] = 1; --h;
                   for (i = 1; i <= m; ++i) (printf("%d ", a[i]));
                   printf(" \n"); ++w;}
            if (a[h] == 3) {
               x = m - h; a[h] = 2;
               if (x > 3) {
                  while (x > 0){++h; a[h] = 2; x -=2;}
                  if    (x == 0){m = h + 1;}
                 else  {m = h;}
                 } // x>3
          else if (x == 3) {
                   for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                   printf(" 2 2"); printf("\n");
                   for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                   printf(" 2 1 1"); printf(" \n");
                   for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                   printf(" 1 1 1 1"); printf("\n");
                   a[h] = 1; --h; m += 2; w += 3;}
          else if (x == 1) {
                   for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                   printf(" 2"); printf(" \n");
                   for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                   printf(" 1 1"); printf("\n");
                   a[h] = 1; --h, m += 2;  w += 2;}
          else if (x == 2) {
```

```
                for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                printf(" 2 1"); printf(" \n");
                for (i = 1; i <= h; ++i) (printf("%d ", a[i]));
                printf(" 1 1 1"); printf(" \n");
                a[h] = 1; --h; m += 2; w += 2;}
            else { //x = 0  m = h
                ++m;
                for (i = 1; i <= m; ++i) (printf("%d ", a[i])); printf("\n");
                a[h] = 1; --h; ++m; ++w;}
           ++w;
           for (i = 1; i <= m; ++i) (printf("%d ", a[i])); printf("\n");
           } // a[h] == 3)
    else if (a[h] > 3) {
          t = m-h+1; a[h] -=1; r = a[h];
          while (t >= r){++h; a[h] = r; t -=r;}
          if     (t < 2) {m = h + t;}
          else {m = h + 1; ++h; a[h] = t;}
          for (i = 1; i <= m; ++i) (printf("%d ", a[i])); printf("\n");
          ++w;
          } // (a[h] > 3)
        }  // while a[1] >= x1
    if  (a[1] < x1) {
       if (a[1] > 1) { --w;}}

    clock2 = clock();
    cpu_time_used = ( (float) (clock2-clock1))/CLOCKS_PER_SEC;
    printf("\n cpu_time_used : %lf seconds \n", cpu_time_used);
    printf("  (CLOCKS_PER_SEC:%d) \n",CLOCKS_PER_SEC);
    printf("\n Start  : %s", ctime(&tstart));
    time(&tfinish);
    printf("\n Finish : %s", ctime(&tfinish));
    printf("\n partitions_W: %lld\n", w);
    scanf ("%d", &i);
    return 0;
}
```

Z3 C code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

int c,c1,m,n,i,h,k,t,r,x,x1,x2,x3,y1,y2;
long long int w,w1,w2,w3,w4,v;
float x4,u1,u2,u3,u4;
clock_t clock1, clock2;
time_t tstart, tfinish;
float cpu_time_used;

int main(void) {
   printf("\n Algorithm Z3 - Multi-Thread setting \n");
   printf("\n Please enter the value of integer N     : ");
   scanf ("%d", &n);
   printf("\n Please enter the lowest  leftmost part  : ");
   scanf ("%d", &x1);
   printf("\n Please enter the highest leftmost part  : ");
   scanf ("%d", &x2);
   printf("\n Please enter the number of Multi-Threads: ");
   scanf ("%d", &x3);
   printf("\n Please enter the Partitions factor      : ");
   scanf ("%f", &x4);

   printf("------------------ Finding Threads with Partitions Borders  ------------------\n");

   if (x1 == 1) {x1 = 2;}
   int a[n+1];
   for (i=0; i<=n; ++i) a[i]=1;
   a[1] = x2 + 1; h = 1; m = n - x2;

   time(&tstart);
   clock1 = clock();
   long long int p[n+1];
   for (i=0; i<=n; ++i) p[i]=0;

   // calculating P(n)
   p[0] = 1;
   for (k=1; k<=n; ++k) {
      for (i=k; i<=n; ++i) {p[i] += p[i-k];}}

    clock2 = clock();
    w = p[n];
    printf("\n PN: %lld ", w);
    cpu_time_used = ( (float) (clock2-clock1))/CLOCKS_PER_SEC;
    printf("       cpu_time: %lf s", cpu_time_used);
    printf("  (CLOCKS_PER_SEC:%d) \n\n",CLOCKS_PER_SEC);

    // finding partitions borders of threads

    for (i=0; i<=n; ++i) a[i]=1;
    a[1] = x2 + 1; h = 1; m = n - x2; w1 = 0; c = 0;
    v = w / x3; v = v * x4;

    printf("Maximum Partitions per one Thread %lld \n\n", v);
```

```
    c1 = a[1];
    while (a[1] >= x1){
        if (v > w1) {
          if (a[1] < c1) {w2 = 0, c1 = a[1];}
                while (a[h] == 2) {++m; a[h] = 1; --h;  ++w1; ++w2;}
                if (a[h] == 3) {
                  x = m - h; a[h] = 2;
                  if (x > 3) {
              while (x > 0) {++h; a[h] = 2; x -=2;}
                  if   (x == 0){m = h + 1;}
               else  {m = h;}}
        else if (x == 3) {a[h] = 1; --h; m += 2; w1 += 3; w2 += 3;}
        else if (x == 1) {a[h] = 1; --h, m += 2; w1 += 2; w2 += 2;}
        else if (x == 2) {a[h] = 1; --h; m += 2; w1 += 2; w2 += 2;}
          else {++m; a[h] = 1; --h; ++m; ++w1; ++ w2;}  //x = 0  m = h
         ++w1; ++w2;
         } // a[h] == 3)
    else if  (a[h] > 3) {
                  t = m-h+1; a[h] -=1; r = a[h];
            while (t >= r){++h; a[h] = r; t -=r;}
            if (t < 2) {m = h + t;}
            else {m = h + 1; ++h; a[h] = t;}
            ++w1; ++w2;
            } // (a[h] > 3)
              } // ( v > w1)
        else {
              clock2 = clock();
              cpu_time_used = ( (float) (clock2-clock1))/CLOCKS_PER_SEC;
              ++c; y1 = a[1] + 1; y2 = x2;
              if (n > x2) {y2 = x2 - 1;}
              if (y1 > y2) {y1 = y2;}
              printf("Thread - %d  -  ", c);  printf("Lower  %d  -  ", y1);  printf("Higer  %d  -  ", y2);
              w3 = w1 - w2 - 1; w4 = w4 + w3;
              printf("Partitions: %lld   ", w3);  printf("cpu_time : %lf s \n\n", cpu_time_used);
              for (i=0; i<=n; ++i) a[i]=1;
              x2 = y1; y1 = 0; y2 = 0; w1=0; w2=0; a[1] = x2; h = 1; m = n - x2 + 1;
              }
       } // while (a[1] > x1)

    clock2 = clock();
    cpu_time_used = ( (float) (clock2-clock1))/CLOCKS_PER_SEC;
    ++y1; --x2; ++c;
    printf("Thread - %d  -  ", c); printf("Lower  %d  -  ", y1); printf("Higer  %d  -  ", x2);
    if (c == 1) { --w1; }
    printf(" Partitions: %lld  ", w1);  printf(" cpu_time : %lf s \n\n", cpu_time_used);
    w4 = w4 + w1;
    printf(" Total Partitions: %lld  ", w4); printf("\n Start  : %s", ctime(&tstart));
    time(&tfinish);
    printf("\n Finish : %s", ctime(&tfinish));
    scanf ("%d", &i);
    return 0;
}
```

12. References:

Using google search "zoghbi integer partitions", here are some relevant websites and online references that mainly mention or use Algorithm ZS1 (by Zoghbi & Stojmenović):

1. Book – The Art Of Computer Programming (TAOCP) Volume 4 Fascicle 3 – Donald E. Knuth
2. Book – Handbook of Applied Algorithms – Amina Nayak and Ivan Stojmenovic
3. Book – Multiagent Coordination Enabling Autonomous Logistics – Arne Schults
4. Book – Future Urban Energy System for Buildings: The Pathway Towards Flexibility – Xingxing Zhang
5. Zoghbi Stojmenovic – Fast algorithms for generating integer partitions - International Journal for Computing Math 1998, vol 70, pp 319-332
6. JA Unified Approach to Algorithms Generating Unrestricted and Restricted Integer Compositions and integer Partitions – 2010 Journal of Mathematical Modeling and Algorithms – JD Opdyke
7. Encoding Partitions As Ascending Compositions. Department of Computer Science, University College Cork – December 2005 – Jerome Kelleher
8. Georgia Institute of Technology – Layout-Aware Mixture Preparation Of Biochemical Fluids
9. National Institute of Health (NIH) – Efficient Algorithms for Calculating Epistatic Geomic…
10. Boston College – Julie Documentation
11. Sporadic.stanford.edu – William Stein and Jonathan Bober
12. University of Toronto – Space & aerospace – Efficient Homotopy Continuation algorithms with …
13. University of Toronto – Efficient Numerical Differentiation of Implicitly-Defined Curves for Sparce Systems – David A. Brown and David W. Zingg
14. SCISPACE – NASA
15. Uncit – H2F: a hierarchical Hadoop framework to process Big Data…
16. MDPI – The integer Nucleolus of Direct Simple Games
17. University of Auckland – ResearchSpace – Cristian S. Calude
18. Hal-Inria- On the cross-sectional distribution of portfolio returns
19. MetaCpan – David Landgren
20. www.arthurcarabott.com – music
21. Application in Plant Science (agEcon) – sjart_st0142.pdf
22. CHATGPT artificial Intelligence
23. A NONPARAMETRIC INDEPENDENCE TEST USING RANDOM PERMUTATIONS - By Jes´us E. Garc´ıa†,‡ and Ver´onica A. Gonz´alez-L´opez Universidade Estadual de Campinas (ZS2)
24. Developments from Programming the Partitions Method for a Power Series Expension – Victor Kowalenko
25. Fluidigm2PURC: automated processing and haplotype inference for double-barcoded PCR amplicons1 - Paul D. Blischak2,9, Maribeth Latvis3, Diego F. Morales-Briones4, Jens C. Johnson5,Verónica S. Di Stilio5, Andrea D. Wolfe2, and David C. Tank6,7,8

Additional websites and resources have become available since the preparation of this paper.Ex

Email: azog0513@gmail.com